# Практическая работа №8. Триггеры в MS SQL Server

## Цель работы

Научиться:

- создавать DML-триггеры;
- использовать `inserted` и `deleted`;
- отслеживать изменения данных;
- реализовывать простые бизнес-правила;
- создавать аудит изменений;
- правильно обрабатывать изменения нескольких строк.

# Сценарий

Представим, что `CollegeDB` используется для хранения информации о студентах.

**Администратор базы данных хочет:**

1. сохранять историю изменения среднего балла студентов;
2. запрещать установку некорректного среднего балла.

Для этого используются триггеры.

# Подготовка

Используйте базу:

```
USE CollegeDB;
```

Проверьте таблицу студентов:

```
SELECT *
FROM college.students;
```

# Задание 1. Таблица аудита

**Создайте таблицу:**

```
college.student_changes
```

Она должна хранить:

- ID записи;
- ID студента;
- старый средний балл;
- новый средний балл;
- дату изменения.

Структура:

```
id
student_id
old_average_grade
new_average_grade
change_date
```

`id` должен автоматически увеличиваться.

# Задание 2. Триггер аудита

Создайте триггер:

```
college.trg_Students_AverageGradeAudit
```

Триггер должен срабатывать после изменения таблицы:

```
college.students
```

Необходимо отслеживать изменение поля:

```
average_grade
```

При изменении студентом среднего балла в `college.student_changes` должна появляться запись:

```
ID студента
Старый балл
Новый балл
Дата изменения
```

Для получения старого значения используйте:

```
deleted
```

Для получения нового значения:

```
inserted
```

# Задание 3. Проверка триггера

Измените средний балл одного студента:

```sql
UPDATE college.students
SET average_grade = 4.8
```

```
WHERE id = 1;
```

После этого проверьте журнал:

```
SELECT *
FROM college.student_changes;
```

В журнале должна появиться новая запись.

# Задание 4. Проверка нескольких строк

Теперь измените средний балл сразу у нескольких студентов.

Например:

```
UPDATE college.students
SET average_grade = average_grade + 0.1
WHERE group_id = 1;
```

Проверьте:

```
SELECT *
FROM college.student_changes;
```

Для каждого изменённого студента должна появиться отдельная запись.

Важно: триггер должен корректно работать не только с одной строкой.

Он срабатывает один раз на весь `UPDATE`, а таблица `inserted` может содержать несколько строк.

# Задание 5. Триггер для проверки данных

**Создайте второй триггер:**

```
college.trg_Students_ValidateGrade
```

Он должен срабатывать при добавлении или изменении студента.

**Средний балл должен находиться в диапазоне:**

```
0–5
```

Если пользователь пытается установить:

```
average_grade < 0
```

или

```
average_grade > 5
```

операция должна быть отменена с помощью ошибки.

Для формирования собственной ошибки используйте:

```
THROW
```

# Задание 6. Проверка бизнес-правила

Попробуйте выполнить корректное изменение:

```sql
UPDATE college.students
SET average_grade = 4.5
WHERE id = 2;
```

Операция должна выполниться.

Затем попробуйте:

```sql
UPDATE college.students
SET average_grade = 6
WHERE id = 2;
```

Операция должна быть отклонена.

Проверьте, что значение в таблице не изменилось:

```
SELECT *
FROM college.students
WHERE id = 2;
```

# Задание 7. Разбор работы триггера

Для созданных триггеров определите:

| Триггер | Событие | Используемые данные |
|---|---|---|
| `trg_Students_AverageGradeAudit` | `UPDATE` | `deleted`, `inserted` |
| `trg_Students_ValidateGrade` | `INSERT`, `UPDATE` | `inserted` |

**Объясните своими словами:**

1. Что находилось в `deleted`?
2. Что находилось в `inserted`?
3. Почему при `UPDATE` нужны обе таблицы?
4. Почему триггер должен корректно работать с несколькими строками?

# Итоговое задание

Представьте, что в колледже хотят отслеживать изменения не только среднего балла, но и возраста студентов.

Модифицируйте триггер аудита так, чтобы таблица изменений дополнительно сохраняла:

```
old_age
new_age
```

Теперь журнал должен фиксировать изменения:

- среднего балла;
- возраста.

**Проверьте работу триггера:**

```
UPDATE college.students
SET
    age = age + 1,
    average_grade = average_grade + 0.1
WHERE id = 3;
```

После этого найдите соответствующую запись в журнале.

## Требования к сдаче

Один файл:

```
Практика_Триггеры_ФИО.sql
```

Файл должен содержать:

1. создание таблицы аудита;
2. создание триггера аудита;
3. тестирование одного изменения;
4. тестирование изменения нескольких строк;
5. создание триггера проверки данных;
6. тестирование корректного значения;
7. тестирование некорректного значения;
8. итоговую модификацию триггера.

## Вопросы для самопроверки

1. Что такое DML-триггер?

2. Когда срабатывает триггер?
3. Чем `AFTER` отличается от `INSTEAD OF`?
4. Что такое `inserted`?
5. Что такое `deleted`?
6. Что находится в `inserted` при `UPDATE`?
7. Что находится в `deleted` при `UPDATE`?
8. Может ли `inserted` содержать несколько строк?
9. Почему нельзя писать триггер только с расчётом на одну изменённую строку?
10. Для чего можно использовать триггеры?
11. Чем триггер отличается от хранимой процедуры?
12. В каких случаях вместо триггера лучше использовать `CHECK` или другой механизм ограничения?

## Что должно быть создано

**В базе должны появиться:**

```
college.student_changes

college.trg_Students_AverageGradeAudit
college.trg_Students_ValidateGrade
```

## Итог

В результате работы вы создали триггеры, которые автоматически реагируют на изменение данных.